

Introduction

Sungyong Park, Ph.D

Data-Intensive Computing and Systems (DISCOS) Laboratory

Department of Computer Science and Engineering, Sogang University

<https://discos.sogang.ac.kr>

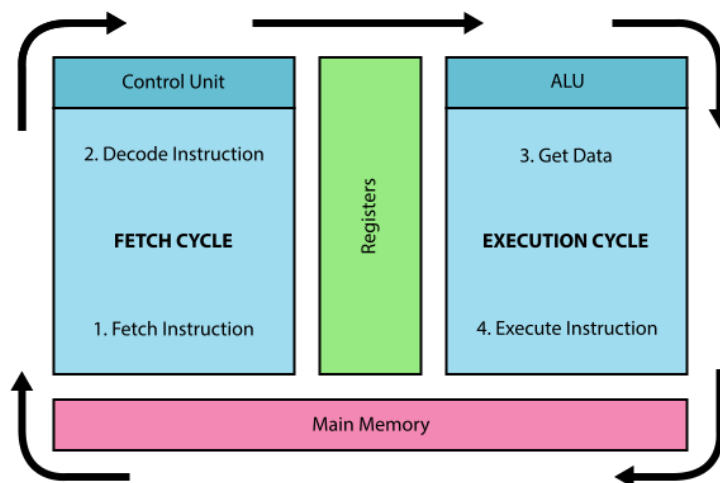
Acknowledgement: Modified from lecture slides made by Prof. Youjip Won (KAIST)
based on the Operating Systems: Three Easy Pieces (OSTEP) textbook

1. Introduction to Operating Systems

Operating System: Three Easy Pieces

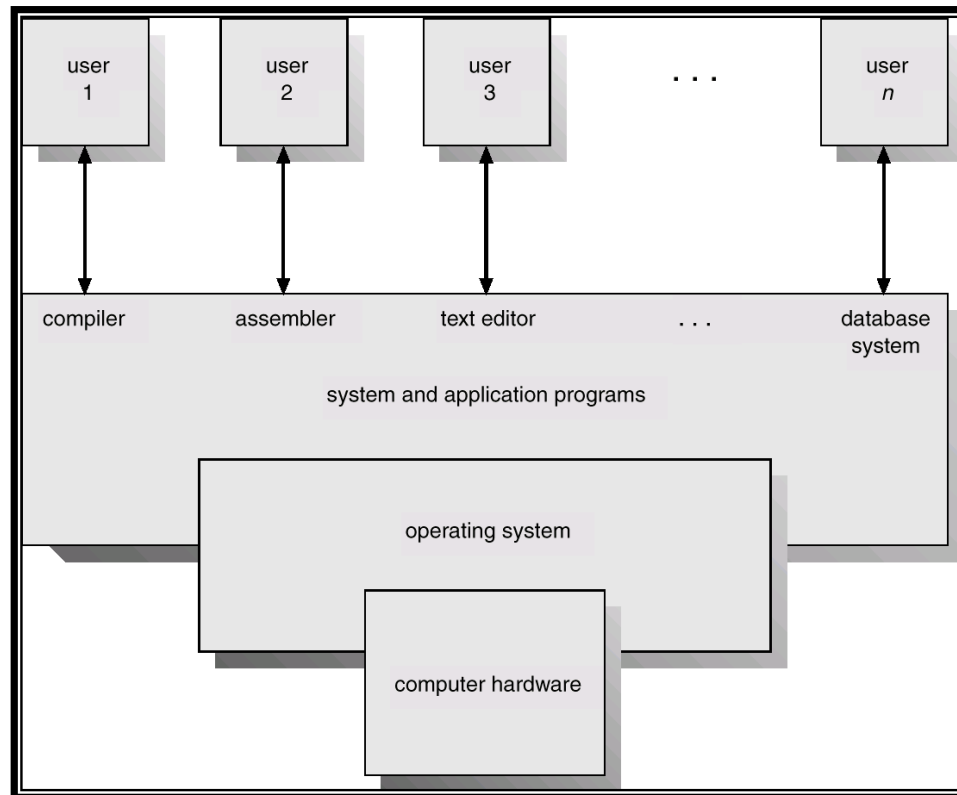
What Happens When a Program Runs?

- ▣ A running program executes instructions.
 1. **Fetch Instruction:** The processor fetches an instruction from memory.
 2. **Decode Instruction:** Figure out which instruction this is.
 3. **Get Data:** Fetch operands if needed.
 4. **Execute Instruction:** i.e., add two numbers, access memory, check a condition, jump to function, and so forth.
 5. The processor moves on to the **next instruction** and so on.



What is a Operating System (OS)?

- A program that acts as an intermediary between a user of a computer (or a program) and the computer hardware.



What OS Does

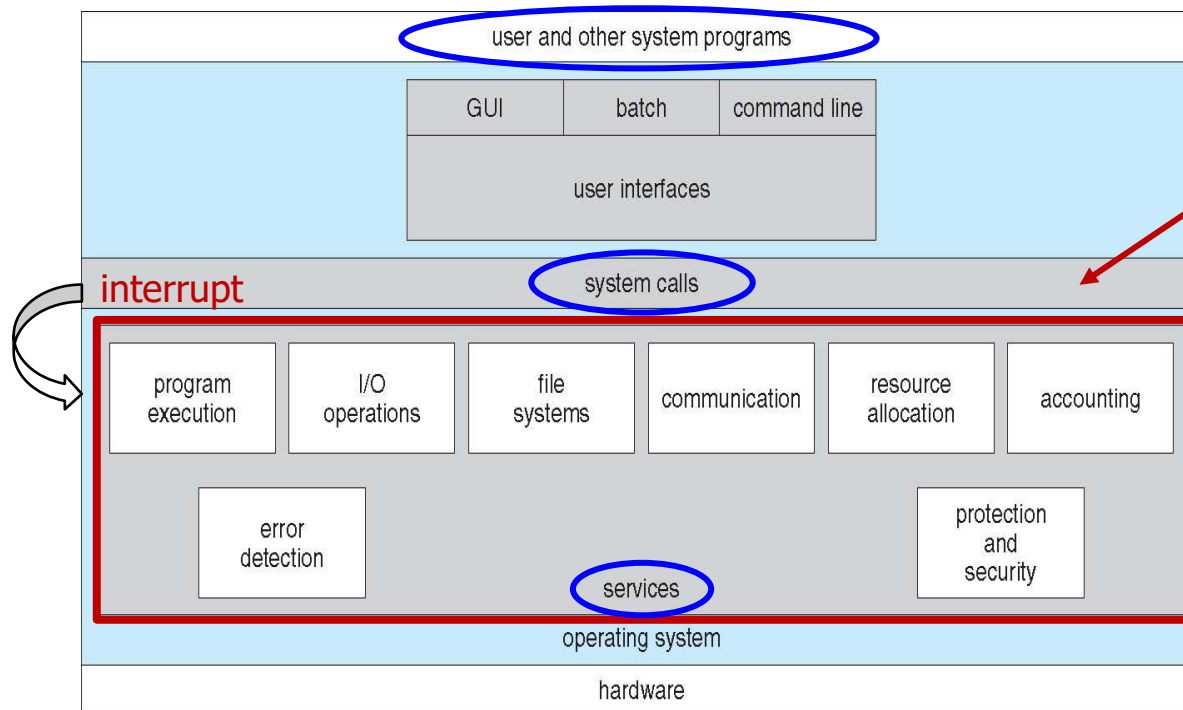
- ▣ OS is responsible for
 - ◆ Making it easy to **run** programs.
 - ◆ Allowing programs to **share** memory.
 - ◆ Enabling programs to **interact** with devices

**OS is in charge of making sure the system operates
correctly and efficiently.**

- ▣ OS provides various services for programs to run.

How to Use OS Services

- ▣ **System call** allows users to tell the OS what to do (services).
 - ◆ The OS provides some interface (APIs, standard library).
 - ◆ A typical OS exports a few hundred system calls.
 - Run programs, access memory, access devices, etc.



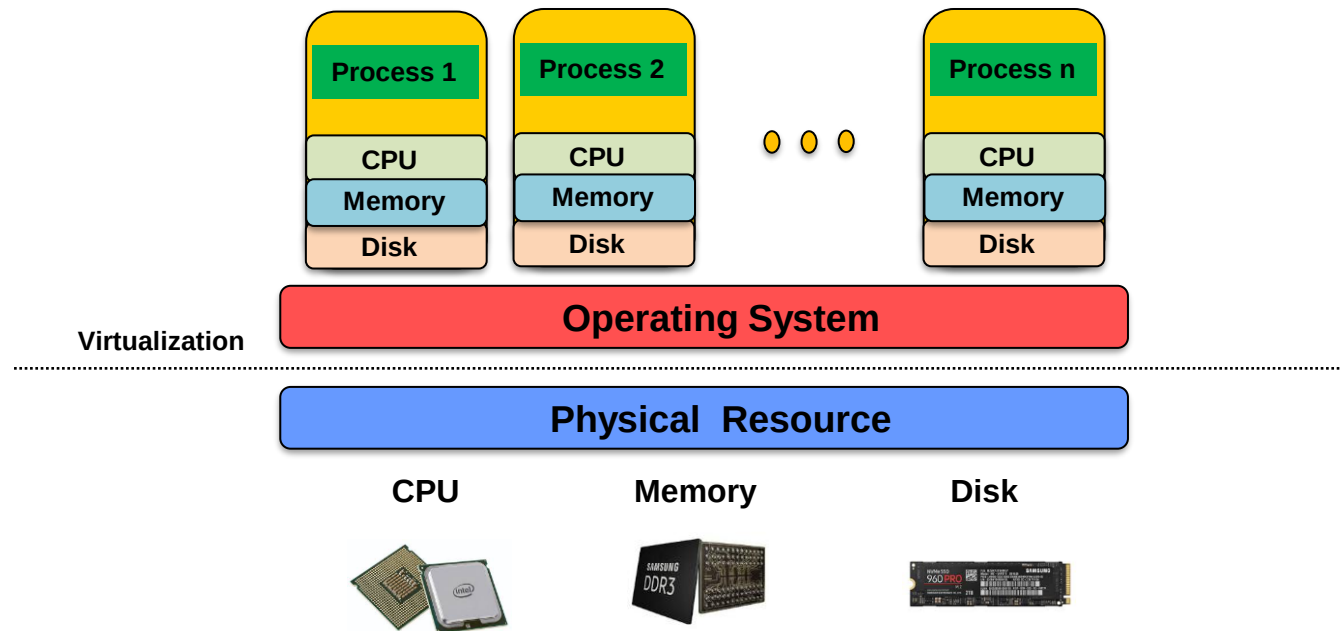
Application Programming Interface (API) typically written in a high-level language (C or C++) and provided as a **library** (e.g., **POSIX API** or **Win32 API**).

The OS is a Resource Manager

- ▣ The OS **manage resources** such as CPU, memory and disk.
- ▣ The OS allows
 - ◆ Many programs to run → **Sharing** the CPU
 - ◆ Many programs to concurrently access their own instructions and data → **Sharing** memory
 - ◆ Many programs to access devices → **Sharing** disks
- ▣ To implement sharing, the OS virtualizes physical resources.

The OS is a Virtual Machine

- ▣ The OS takes a **physical resource** and transforms it into a **virtual form** of itself.
 - ◆ **Physical resource:** Processor, Memory, Disk ...
 - ◆ The virtual form is more general, powerful and easy-to-use.



Virtualizing the CPU

- ▣ The system has a very large number of virtual CPUs.
 - ◆ Turning a single CPU into a seemingly infinite number of CPUs.
 - ▣ Allowing many programs to seemingly run at once.
 - ◆ Program executions are interleaved.
 - ◆ Each program seems to have exclusive use of the CPU.
- **Virtualizing the CPU**

Example of Virtualizing the CPU (cpu.c)

- A program that loops and prints (cpu.c)

```
1      #include <stdio.h>
2      #include <stdlib.h>
3      #include <sys/time.h>
4      #include <assert.h>
5      #include "common.h"           // Contains Spin() function
6
7      int
8      main(int argc, char *argv[])
9      {
10         if (argc != 2) {
11             fprintf(stderr, "usage: cpu <string>\n");
12             exit(1);
13         }
14         char *str = argv[1];
15         while (1) {
16             Spin(1); // Repeatedly checks the time and
                       // returns once it has run for a second
17             printf("%s\n", str);
18         }
19         return 0;
20     }
```

Let's Run (Case 1)

▣ Execution result 1.

```
prompt> gcc -o cpu cpu.c -Wall
prompt> ./cpu "A"
A
A
A
^C
prompt>
```

Run forever; Only by pressing “Control-c” can we halt the program

Let's Run (Case 2)

■ Execution result 2.

Run as a background job !

```
prompt> ./cpu A & ./cpu B & ./cpu C & ./cpu D &
[1] 7353
[2] 7354
[3] 7355
[4] 7356
A
B
D
C
A
B
D
C
A
C
B
D
...
```

If several programs want to run at a particular time, which should run?

Based on a policy of the OS (scheduling algorithm)

Even though we have only **one processor**, all four of programs seem to be running **at the same time** ! → **OS virtualizes the CPU** !

Virtualizing Memory

- ▣ The physical memory is an array of bytes.
 - ▣ A program keeps all of its data structures in memory.
 - ◆ **Read memory** (load):
 - Specify an address to be able to access the data.
 - ◆ **Write memory** (store):
 - Specify the data to be written to the given address.
 - ▣ Each program has its own private memory.
 - ◆ Each program seems to have exclusive use of main memory.
- **Virtualizing memory**

Example of Virtualizing Memory (mem.c)

■ A program that accesses memory (mem.c)

```
1      #include <unistd.h>
2      #include <stdio.h>
3      #include <stdlib.h>
4      #include "common.h"
5
6      int
7      main(int argc, char *argv[])
8      {
9          int *p = malloc(sizeof(int));    // a1: allocate memory
10         assert(p != NULL);
11         printf("(%d) address of p: %08x\n",
12                getpid(), (unsigned) p); // a2: print out the
13                                         // address of the memory
14                                         // a3: put zero into the
15                                         // first slot of the memory
16         *p = 0;
17
18         while (1) {
19             Spin(1);
20             *p = *p + 1;
21             printf("(%d) p: %d\n", getpid(), *p); // a4
22         }
23         return 0;
24     }
```

Get process id : will be covered later !

Let's Run (Case 1)

- ▣ The output of the program `mem.c`

```
prompt> ./mem
(2134) memory address of p: 00200000
(2134) p: 1
(2134) p: 2
(2134) p: 3
(2134) p: 4
(2134) p: 5
^C
```

- ◆ The newly allocated memory is at address `00200000`.
- ◆ It updates the value and prints out the result.

Let's Run (Case 2)

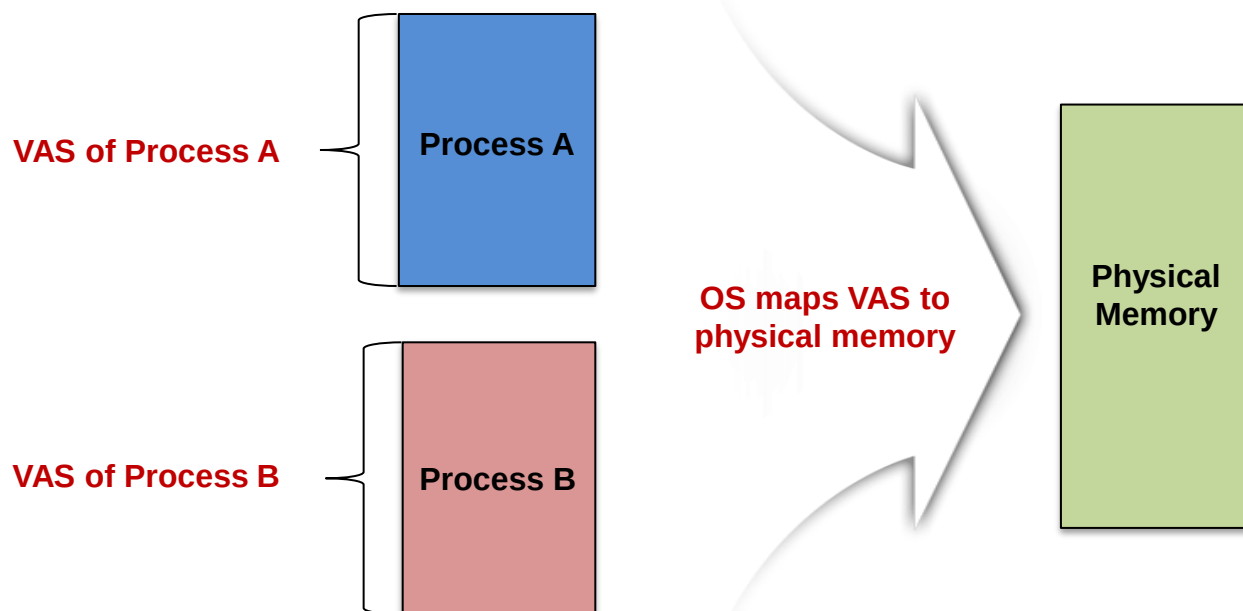
▣ Running `mem.c` multiple times

```
prompt> ./mem & ./mem &  
[1] 24113  
[2] 24114  
(24113) memory address of p: 00200000  
(24114) memory address of p: 00200000  
(24113) p: 1  
(24114) p: 1  
(24114) p: 2  
(24113) p: 2  
(24113) p: 3  
(24114) p: 3  
...
```

- ◆ Each program runs as if it has its **own private memory**.
 - Each running program has allocated memory at the same address.
 - Each seems to be updating the value at `00200000` independently.

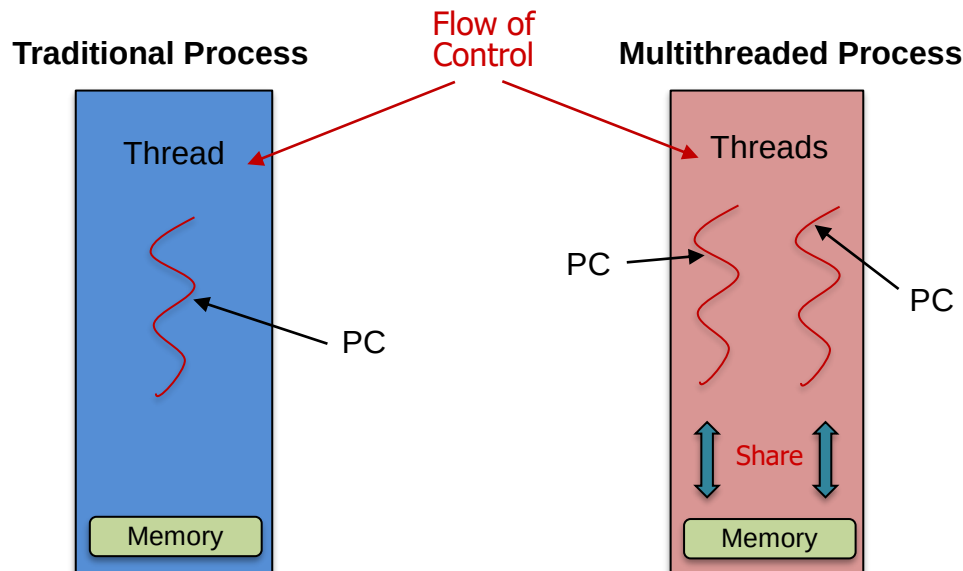
Virtual Address Space

- ▣ Each process accesses its own private **virtual address space (VAS)**.
 - ◆ The OS maps **virtual address space** onto the **physical memory**.
 - ◆ A memory reference within one running program does not affect the address space of other processes.
 - ◆ Physical memory is a shared resource, managed by the OS.



The Problem of Concurrency

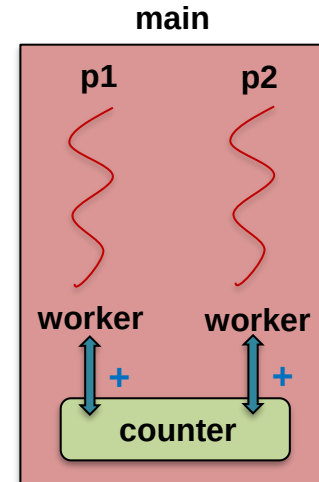
- The OS is juggling **many things at once**, first running one process, then another, and so forth.
- Modern **multi-threaded programs** also exhibit the concurrency problem.
 - ◆ Multi-threads are flow of controls within a process that can be running at the same time. → Details will be covered later !



Example of Concurrency Problem (thread.c)

■ A Multi-threaded Program (thread.c)

```
1      #include <stdio.h>
2      #include <stdlib.h>
3      #include "common.h"
4
5      volatile int counter = 0; int loops;
6
7      void *worker(void *arg) {
8          int i;
9          for (i = 0; i < loops; i++) counter++;
10         return NULL;
11     }
12
13     int main(int argc, char *argv[]) {
14         if (argc != 2) {
15             fprintf(stderr, "usage: threads <value>\n"); exit(1);
16         }
17         loops = atoi(argv[1]);
18         pthread_t p1, p2;
19         printf("Initial value : %d\n", counter);
20
21         Pthread_create(&p1, NULL, worker, NULL);
22         Pthread_create(&p2, NULL, worker, NULL);
23         Pthread_join(p1, NULL);
24         Pthread_join(p2, NULL);
25         printf("Final value : %d\n", counter);
26         return 0;
27     }
```



Let's Run

- ▣ `loops` determines how many times each of the two workers will increment the shared counter in a loop.

- ◆ `loops: 1000.`

```
prompt> gcc -o thread thread.c -Wall -pthread
prompt> ./thread 1000
Initial value : 0
Final value : 2000
```

- ◆ `loops: 100000.`

```
prompt> ./thread 100000
Initial value : 0
Final value : 143012 // huh??
prompt> ./thread 100000
Initial value : 0
Final value : 137298 // what the??
```

Why is This Happening?

- ▣ Increment a shared counter ($counter++$) $\rightarrow$ takes three instructions.
 1. Load the value of the counter from memory into register ($reg \leftarrow memory$).
 2. Increment it ($reg \leftarrow reg + 1$).
 3. Store it back into memory ($memory \leftarrow reg$).

- ▣ These three instructions do not execute **atomically** (i.e., interleave).
 $\rightarrow$ Problem of **concurrency** happen.

- ▣ OS provides a mechanism to handle this problem.

Persistence

- ❑ Devices such as DRAM store values in a **volatile** manner.
- ❑ Hardware and software are needed to store data **persistently**.
 - ◆ Hardware: I/O device such as hard drives and solid-state drives (SSDs).
 - ◆ Software:
 - **File system** manages the disk.
 - File system is responsible for storing any **files** the user creates in a reliable and efficient manner on the disks of the system.
- ❑ Unlike the CPU and memory, the OS does not create a private, virtualized disk for each application.
- ❑ Rather it is assumed that users will want to share information that is in files.

Example of Persistence

- Create a file (`/tmp/file`) that contains the string “hello world”.

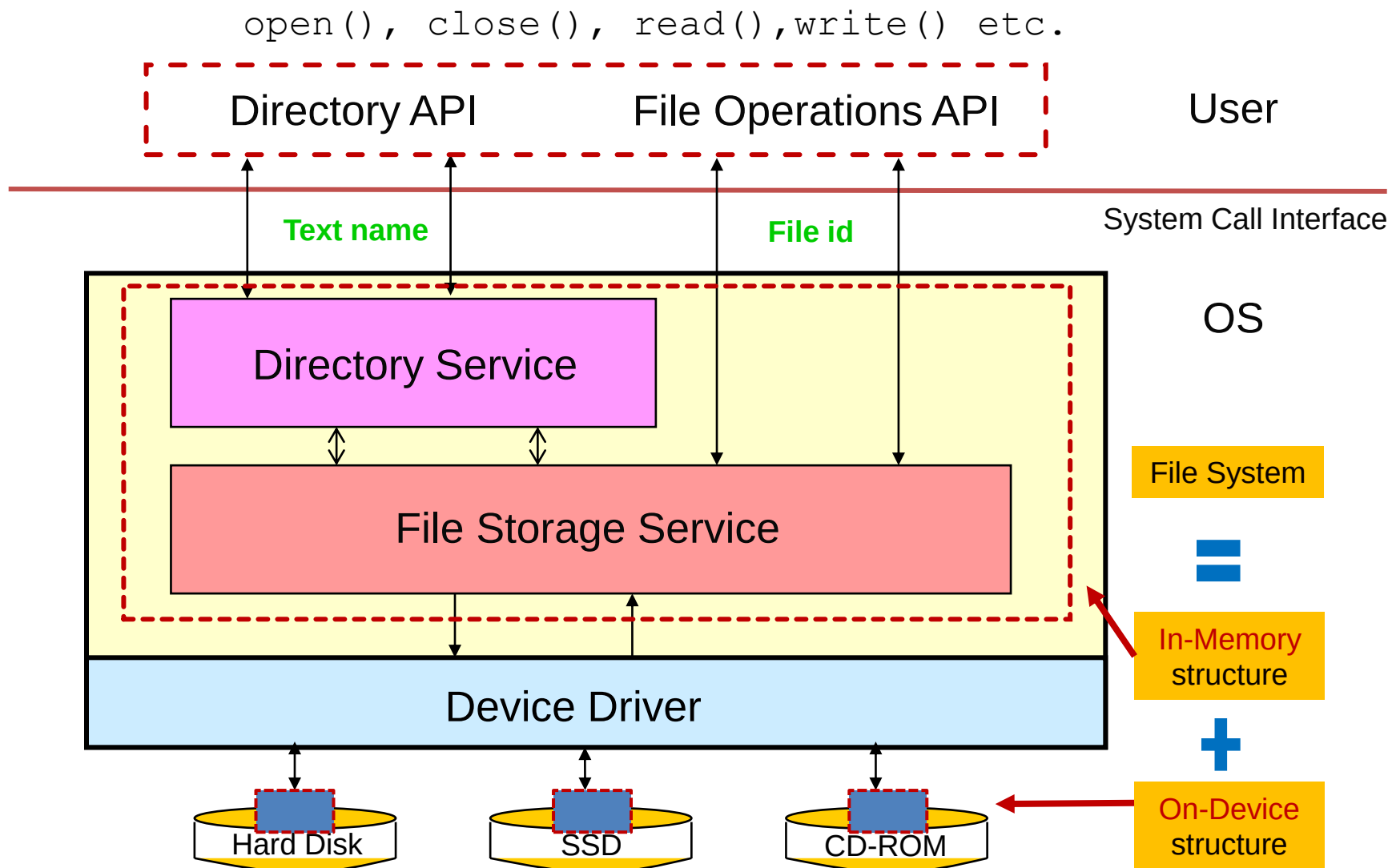
```
1      #include <stdio.h>
2      #include <unistd.h>
3      #include <assert.h>
4      #include <fcntl.h>
5      #include <sys/types.h>
6
7      int
8      main(int argc, char *argv[])
9      {
10         int fd = open("/tmp/file", O_WRONLY | O_CREAT
                        | O_TRUNC, S_IRWXU);
11         assert(fd > -1);
12         int rc = write(fd, "hello world\n", 13);
13         assert(rc == 13);
14         close(fd);
15         return 0;
16     }
```

`open()`, `write()`, and `close()` system calls are routed to the part of OS called the **file system**, which handles the requests.

What OS Does for Persistence?

- ▣ What OS does in order to write to disk?
 - ◆ Figure out **where** on disk this new data will reside.
 - ◆ **Issue I/O** requests to the underlying storage device with a **device driver**.
- ▣ File system handles system crashes during write.
 - ◆ **Journaling** or **copy-on-write**.
 - ◆ Carefully ordering writes to disk to ensure that if a failure occurs during the write sequence, the system can recover to reasonable state afterwards.

What is File System?



OS Design Goals

- ❑ Build up **abstraction**
 - ◆ Make the system convenient and easy to use.
- ❑ Provide high **performance**
 - ◆ Minimize the overhead of the OS.
 - ◆ OS must strive to provide virtualization without excessive overhead.
- ❑ **Protection** between applications
 - ◆ Isolation: Bad behavior caused by one process does not harm others and the OS itself.
- ❑ High degree of **reliability**
 - ◆ The OS must also run non-stop.
- ❑ Other issues
 - ◆ **Energy-efficiency, Security, Mobility**

History of OS – Early OS

- Initially, the OS didn't do too much → just a set of libraries (procedure call) of commonly used functions (i.e., I/O etc.).
 - ◆ Mainly used for **batch** processing jobs and used as a **single** user OS.
- The OS started to provide a **protection** mechanism by separating user mode vs kernel mode operations.
 - ◆ User invokes system calls in **user mode** and the control is transferred to the **kernel mode** via trap (interrupt).



Main frame computers

History of OS – Era of Multiprogramming

- ❑ **Multiprogramming** became commonplace due to the desire to make better use of machine resources.
 - ◆ Issues such as **memory protection** and **concurrency** became important.
 - ◆ One of the major practical advances this era was the introduction to the **UNIX** OS, primarily thanks to Ken Thompson (and Dennis Richie) at Bell Labs.



Mini computer (e.g., PDP-11)

Introduction of Unix OS

History of OS – Modern Era

- OS is everywhere !



What Lies All These Behind...

